



UNIDAD 2

Algoritmo de Trayectoria.

TEMA 1:

Fundamentos de Algoritmos de Trayectoria.

Descripción:

Los algoritmos de trayectoria comprenden una familia de métodos metaheurísticos que exploran el espacio de soluciones siguiendo una única trayectoria individual, a diferencia de las técnicas que operan con poblaciones. Su mecanismo se fundamenta en la definición de vecindarios, los cuales permiten transitar de una solución candidata a otra mediante movimientos locales, con el objetivo de mejorar progresivamente el valor de una función objetivo.

Al finalizar la unidad, el estudiante estará en capacidad de comprender los fundamentos conceptuales de los algoritmos de trayectoria, implementarlos en problemas concretos y evaluar su rendimiento en comparación con otros métodos metaheurísticos.

Metaheurísticas

PERIODO ACADÉMICO 2026-1S

TABLA DE CONTENIDO

Tema 1. Fundamentos de Algoritmos de Trayectoria.	2
1.1. Definición y enfoque de búsqueda basado en trayectoria individual.	2
1.2. Análisis teórico del espacio de búsqueda y estructuras de vecindad.	3
1.3. Clasificación General de los algoritmos.	4
1.4. Parámetros principales, comportamiento y mecanismos de escape.	12
Bibliografía	14

Tema 1. Fundamentos de Algoritmos de Trayectoria.

1.1. Definición y enfoque de búsqueda basado en trayectoria individual.

Los algoritmos de trayectoria son técnicas de optimización que se enfocan en la exploración del espacio de soluciones mediante el recorrido de una trayectoria única.

A diferencia de los algoritmos bioinspirados poblacionales, que trabajan con múltiples candidatos a la vez, los de trayectoria se concentran en mejorar progresivamente una solución individual, desplazándose hacia soluciones vecinas en busca de un mejor resultado (Rowold et al., 2022).

Su fortaleza radica en la simplicidad y eficiencia computacional, ya que requieren menos memoria y pueden ofrecer soluciones competitivas en tiempos de cálculo reducidos. Sin embargo, enfrentan el desafío de caer en óptimos locales, por lo que incorporan mecanismos de escape como la aleatoriedad controlada o el registro de movimientos previos.

Estos algoritmos resultan útiles en aplicaciones donde la rapidez y la adaptabilidad son esenciales, como la planificación de rutas, la asignación de recursos o la robótica, ya que permiten obtener soluciones viables en escenarios donde los métodos exactos resultan impracticables (Hu et al., 2023).

Definición

Análisis

Clasificación

Parámetros

 Ejemplo:

- El Simulated Annealing (SA) se inspira en el proceso físico de enfriamiento de los metales. En lugar de aceptar únicamente mejoras inmediatas, el algoritmo permite, con cierta probabilidad, aceptar soluciones peores al inicio de la búsqueda. Esto evita quedar atrapado en óptimos locales y, conforme avanza el proceso (al disminuir la "temperatura"), la exploración se concentra en los mejores entornos. Un uso común del SA es en la optimización de la planificación de rutas de distribución, donde ayuda a reducir costos y tiempos logísticos.

1.2. Análisis teórico del espacio de búsqueda y estructuras de vecindad.

El estudio de los algoritmos de trayectoria se basa en el análisis del espacio de búsqueda y de las estructuras de vecindario que lo organizan.

En estos métodos, una solución inicial evoluciona mediante movimientos hacia soluciones vecinas, guiados por reglas de exploración predefinidas. El espacio de búsqueda, que comprende todas las soluciones posibles del problema, varía en complejidad según su dimensionalidad y restricciones. La efectividad de estos algoritmos depende crucialmente de su capacidad para navegar este espacio, equilibrando la exploración global con la explotación local de regiones prometedoras.

El núcleo de su funcionamiento reside en la definición de los vecindarios, que determinan qué soluciones son accesibles desde un punto dado. Por ejemplo, en problemas como el del viajante (TSP), un vecindario puede construirse mediante operaciones de intercambio o inversión de nodos.

Definición

Análisis

Clasificación

Parámetros

La elección de esta estructura afecta directamente la eficiencia del algoritmo, influyendo tanto en la diversidad de soluciones exploradas como en su capacidad para evitar óptimos locales. Teóricamente, estos métodos pueden entenderse como procesos de búsqueda guiada —ya sea estocástica o determinista—, lo que permite evaluar propiedades clave como su convergencia, cobertura del espacio y coste computacional. A diferencia de los enfoques poblacionales, que dependen de la diversidad inherente a múltiples soluciones, los algoritmos de trayectoria destacan por su eficiencia memoria y su reliance crítico en el diseño inteligente de vecindarios y reglas de transición para alcanzar soluciones competitivas en problemas de optimización complejos.

1.3. Clasificación General de los algoritmos.

Los algoritmos de trayectoria se clasifican principalmente según la manera en que gestionan la exploración del espacio de soluciones y los mecanismos que emplean para escapar de los óptimos locales (Gamayunov et al., 2023).

1.3.1 Simulated Annealing (SA).

El recocido simulado (SA) es un algoritmo metaheurístico basado en trayectorias, utilizado recientemente para resolver problemas de optimización e inspirado en el proceso físico de recocido en metalurgia, en el cual un material es calentado a altas temperaturas y luego enfriado de manera gradual y controlada. Durante la fase de alta temperatura, el algoritmo permite movimientos aleatorios que potencialmente aceptan soluciones peores que la actual, lo que le confiere una capacidad de escape de óptimos locales. A medida que el parámetro de "temperatura" disminuye de forma progresiva, la búsqueda se vuelve más selectiva, favoreciendo los movimientos que mejoran la solución y estabilizando el sistema hacia un estado óptimo o cercano al óptimo (Guilmeau et al., 2022).

Definición

Análisis

Clasificación

Parámetros

Estructura Básica del Algoritmo Simulated Annealing

- **Inicialización**

- Se establece una solución inicial (x_0) y una temperatura inicial (T_0)

- **Búsqueda de vecinos**

- A partir de la solución actual se busca una solución vecina.

- **Criterio de aceptación**

- Si la nueva solución mejora la función objetivo se acepta para el siguiente criterio de aceptación.
- Si no mejora, puede aceptarse con probabilidad en base a esta fórmula, donde T es la temperatura:

$$P = e^{-\Delta E/T} \quad \text{donde} \quad \Delta E = f(x') - f(x)$$

- **Enfriamiento**

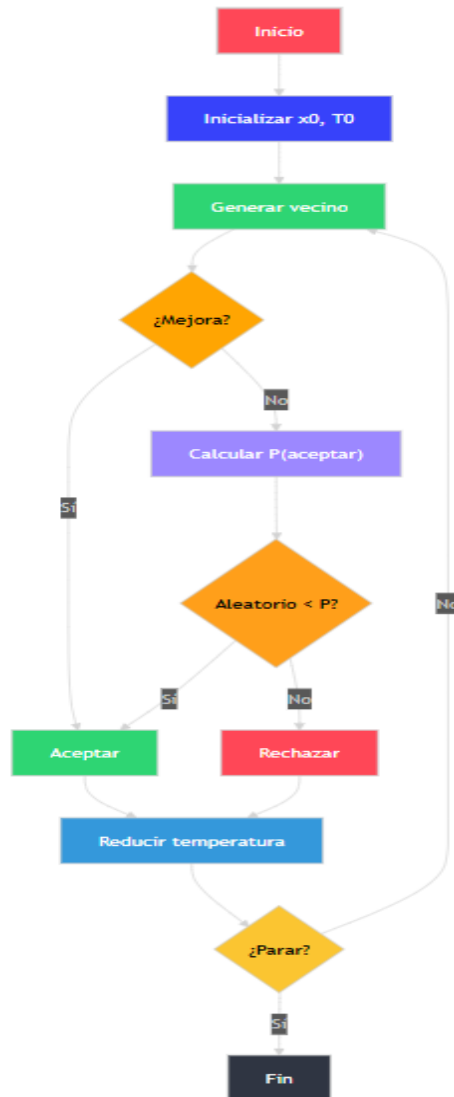
- Se reduce la temperatura gradualmente según un schedule:

$$T_{k+1} = \alpha \cdot T_k, \quad 0 < \alpha < 1$$

- **Iteración**

- Se repite hasta que T alcance un valor mínimo o se cumpla un número máximo de iteraciones.

Figura 1. Diagrama de flujo del algoritmo de Recocido Simulado (Simulated Annealing)



Fuente: Elaboración Propia del autor.

● Ejemplo de Implementación en python

A continuación, se presenta la implementación paso a paso, donde se inicializan los parámetros, se genera una solución vecina perturbando la solución actual con ruido gaussiano, y se aplica el criterio de aceptación de Metrópolis para decidir transiciones entre soluciones, mientras la temperatura disminuye gradualmente en cada iteración:

```
1  import numpy as np
2
3  def sphere(x):
4      return np.sum(x**2)
5
6  # Parámetros
7  n_iter = 1000
8  dim = 2
9  T = 100.0
10 alpha = 0.99
11
12 # Solución inicial
13 current = np.random.uniform(-5, 5, dim)
14 current_eval = sphere(current)
15 best, best_eval = current, current_eval
16
17 for i in range(n_iter):
18     # Generar vecino
19     candidate = current + np.random.normal(0, 1, dim)
20     candidate_eval = sphere(candidate)
21
22     # Calcular diferencia de energía
23     delta = candidate_eval - current_eval
24
25     # Aceptación
26     if delta < 0 or np.random.rand() < np.exp(-delta / T):
27         current, current_eval = candidate, candidate_eval
28         if candidate_eval < best_eval:
29             best, best_eval = candidate, candidate_eval
30
31     # Enfriamiento
32     T *= alpha
33
34 print("Mejor solución encontrada:", best)
35 print("Valor de la función:", best_eval)
```

Fuente: Elaboración Propia del autor.

1.3.2. Búsqueda Tabú (Tabu Search, TS).

La Búsqueda Tabú (TS) se distingue de otros algoritmos metaheurísticos como PSO, GA o ACO al no inspirarse en sistemas naturales, sino en una metáfora cognitiva basada en la memoria humana. Su objetivo principal es evitar ciclos repetitivos durante la optimización, donde un algoritmo podría caer en movimientos o soluciones previamente exploradas. Para lograrlo, TS incorpora una memoria prohibitiva, conocida como "lista tabú", que registra soluciones o movimientos recientes para impedir que el algoritmo regrese a ellos temporalmente. Este mecanismo no solo permite escapar de óptimos locales, sino que también facilita una exploración inteligente del espacio de búsqueda al dirigir la búsqueda hacia regiones inexploradas. La premisa fundamental de TS es que "recordar dónde se ha estado" evita retrocesos innecesarios y abre paso a trayectorias más prometedoras y eficientes (Venkateswarlu, 2021).

Además, la versatilidad de TS ha permitido su aplicación en problemas reales de alta complejidad, como el enrutamiento de vehículos en contextos de emergencia. Por ejemplo, se ha empleado en la optimización de rutas heterogéneas con recogida dividida en escenarios de evacuación y logística, demostrando su capacidad para adaptarse a condiciones dinámicas y críticas (Xu et al., 2022).

Estructura Básica del Algoritmo

Inicialización

- El algoritmo comienza definiendo una solución inicial, que puede ser generada aleatoriamente o mediante heurísticas simples. Simultáneamente, se crea una lista tabú vacía, la cual actuará como memoria de corto plazo para registrar movimientos o situaciones recientes y evitar ciclos repetitivos.

Generación de vecinos

- En cada iteración, se genera un conjunto de soluciones vecinas aplicando pequeñas modificaciones o movimientos locales a la solución actual. Estos movimientos pueden incluir intercambios, inserciones o perturbaciones específicas según el problema.

Definición

Análisis

Clasificación

Parámetros

Selección del mejor vecino

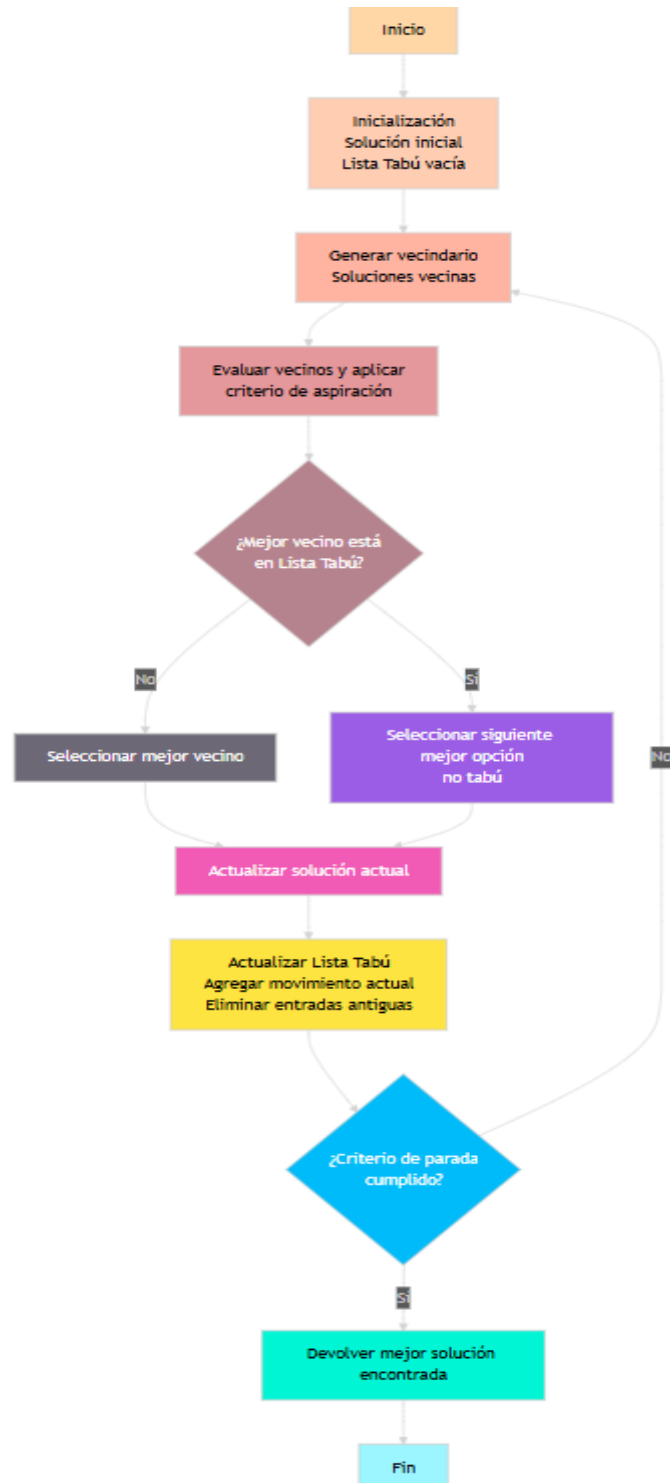
- Se evalúan todas las soluciones vecinas y se selecciona la mejor que no esté en la lista tabú. No obstante, si un movimiento tabú cumple un criterio de aspiración (como mejorar la mejor solución global encontrada), se acepta excepcionalmente.

Actualización

- La solución elegida se convierte en la nueva solución actual. La lista tabú se actualiza agregando los movimientos recientes realizados, y si la lista alcanza su tamaño máximo, se eliminan los elementos más antiguos para mantener su capacidad limitada.

Iteración

- El proceso se repite hasta cumplir un criterio de parada (número máximo de iteraciones o estancamiento). La lista tabú asegura diversificación al evitar recurrir a soluciones recientes, mientras el criterio de aspiración preserva la intensificación.

Figura 2. Diagrama de flujo del algoritmo de Búsqueda Tabú (Tabu Search)

Fuente: Elaboración Propia del autor

Implementación en Python.

El siguiente código implementa el algoritmo de Búsqueda Tabú (Tabu Search) aplicado a la optimización de la función sphere, una función benchmark clásica. El criterio de aspiración se aplica implícitamente al siempre elegir el mejor vecino disponible, permitiendo así explorar nuevas regiones del espacio de soluciones de manera inteligente y eficiente.

```
1 import numpy as np
2
3 def sphere(x):
4     return np.sum(x**2)
5
6 # Parámetros
7 n_iter = 200
8 dim = 2
9 tabu_tenure = 10 # tamaño de la lista tabú
10 step_size = 0.5
11
12 # Inicialización
13 current = np.random.uniform(-5, 5, dim)
14 best = np.copy(current)
15 tabu_list = []
16
17 for it in range(n_iter):
18     # Generar vecinos
19     neighbors = [current + np.random.uniform(-step_size, step_size, dim) for _ in range(20)]
20     neighbors = [n for n in neighbors if list(n) not in tabu_list]
21
22     # Evaluar vecinos
23     fitness = [sphere(n) for n in neighbors]
24     best_neighbor = neighbors[np.argmin(fitness)]
25
26     # Actualizar solución actual
27     current = best_neighbor
28
29     # Actualizar lista tabú
30     tabu_list.append(list(current))
31     if len(tabu_list) > tabu_tenure:
32         tabu_list.pop(0)
33
34     # Actualizar mejor global
35     if sphere(current) < sphere(best):
36         best = np.copy(current)
37
38 print("Mejor solución encontrada:", best)
39 print("Valor de la función:", sphere(best))
```

Fuente: Elaboración Propia del autor

1.4. Parámetros principales, comportamiento y mecanismos de escape.

Tabla 1. Comparación entre los algoritmos de Recocido Simulado (SA) y Búsqueda Tabú (TS)

Aspecto	Similitud Annealing (SA)	Tabu Search (TS)
Parámetros principales	• Temperatura inicial • Tasa de enfriamiento • Iteraciones por temperatura	• Tamaño de la lista tabú (tenencia) • Número de vecinos generados • Criterio de aspiración
Comportamiento	Comienza con una búsqueda amplia y caótica (alta temperatura) y gradualmente reduce la exploración para centrarse en la explotación.	Realiza una búsqueda sistemática en la vecindad, evitando ciclos y repeticiones gracias al uso de memoria.
Mecanismo de escape de óptimos locales	Acepta soluciones peores con cierta probabilidad decreciente según la temperatura.	Prohíbe volver a soluciones ya visitadas (lista tabú), forzando la exploración de trayectorias alternativas.
Riesgos	• Si la tasa de enfriamiento es muy rápida, puede quedarse atrapado en mínimos locales. • Si es muy lenta, puede ser computacionalmente costoso.	Una lista tabú demasiado corta no evita ciclos; una demasiado larga puede excluir soluciones potencialmente útiles.
Fortalezas	Simple, flexible y aplicable tanto a problemas continuos como discretos.	Muy eficaz en problemas combinatorios

Definición

Análisis

Clasificación

Parámetros

complejos,
especialmente en
grafos, ruteo y
asignación.

Fuente: Elaboración Propia del autor.

Bibliografía

Bahy, M. B., & Musdholifah, A. (2022). Fast Non-dominated Sorting in Multi Objective Genetic Algorithm for Bin Packing Problem. IJCCS (Indonesian Journal of Computing and Cybernetics Systems), 16(1), 55-66. <https://doi.org/10.22146/ijccs.70677>

Bansal, J. C., Bajpai, P., Rawat, A., & Nagar, A. K. (2023). Advancements in the Sine Cosine Algorithm. En J. C. Bansal, P. Bajpai, A. Rawat, & A. K. Nagar (Eds.), Sine Cosine Algorithm for Optimization (pp. 87-103). Springer Nature. https://doi.org/10.1007/978-981-19-9722-8_5

Gamayunov, A., Postnikov, A., & Ferrer, G. (2023). DDPEN: Trajectory Optimisation With Sub Goal Generation Model (No. arXiv:2301.07433). arXiv. <https://doi.org/10.48550/arXiv.2301.07433>

Guilmeau, T., Chouzenoux, E., & Elvira, V. (2022). Proximal-Based Adaptive Simulated Annealing for Global Optimization. ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 5453-5457. <https://doi.org/10.1109/ICASSP43922.2022.9746626>

Hu, T., Ochoa, G., & Banzhaf, W. (2023). Phenotype Search Trajectory Networks for Linear Genetic Programming (No. arXiv:2211.08516). arXiv. <https://doi.org/10.48550/arXiv.2211.08516>

Liu, G., Xu, X., Wang, F., & Tang, Y. (2022). Solving Traveling Salesman Problems Based on Artificial Cooperative Search Algorithm. Computational Intelligence and Neuroscience, 2022(1), 1008617. <https://doi.org/10.1155/2022/1008617>

R, A. K. K., & J, E. R. D. (2023). Mitigation of Make Span Time in Job Shop Scheduling Problem Using Gannet Optimization Algorithm. EAI Endorsed Transactions on Scalable Information Systems, 10(5). <https://doi.org/10.4108/eetsis.2913>

Rowold, M., Ögretmen, L., Kerbl, T., & Lohmann, B. (2022). Efficient Spatiotemporal Graph Search for Local Trajectory Planning on Oval Race Tracks. Actuators, 11(11), 319. <https://doi.org/10.3390/act11110319>

Venkateswarlu, C. (2021). A Metaheuristic Tabu Search Optimization Algorithm: Applications to Chemical and Environmental Processes. En

Definición

Análisis

Clasificación

Parámetros

Engineering Problems—Uncertainties, Constraints and Optimization Techniques. IntechOpen. <https://doi.org/10.5772/intechopen.98240>

Weiss, E., & Kaminka, G. A. (2023). Planning with Dynamically Estimated Action Costs (No. arXiv:2206.04166). arXiv. <https://doi.org/10.48550/arXiv.2206.04166>

Xu, L., Wang, Z., Chen, X., & Lin, Z. (2022). Multi-Parking Lot and Shelter Heterogeneous Vehicle Routing Problem With Split Pickup Under Emergencies. IEEE Access, 10, 36073-36090. <https://doi.org/10.1109/ACCESS.2022.3163715>